C Practice Checklist

From zero to hero: tick each task when it compiles cleanly, handles edge cases, and has no leaks

Rules of the game

- Compile with `gcc -Wall -Wextra -Werror -g`. From Level 4 on, also run with `-fsanitize=address` and Valgrind.
- Check every return value: `scanf`, `malloc`, `fopen`.
- A task is done only when it handles edge cases (empty, 0, negative, INT_MAX, very long input) and leaks nothing.
- Write it yourself first. Look at hints only after a real attempt.
- Redo a task from scratch a week later without looking.

Level	Topic	Tasks	Done
Level 0	Basics	1-8	____ / 8
Level 1	Conditions and Loops	9-24	____ / 16
Level 2	Functions and Arrays	25-38	____ / 14
Level 3	Strings (no string.h)	39-50	____ / 12
Level 4	Pointers and Dynamic Memory	51-63	____ / 13
Level 5	Structs, Enums, Unions, Bits	64-71	____ / 8
Level 6	Data Structures	72-82	____ / 11
Level 7	Files and I/O	83-90	____ / 8
Level 8	Preprocessor, Build Tools, Multi-file	91-97	____ / 7
Level 9	Systems Programming (Linux/Unix)	98-106	____ / 9
Level 10	Hero Projects	107-115	____ / 9

Total: 115 tasks

Level 0: Basics

Goal: get comfortable with compiling, printf/scanf, types, and arithmetic.

	#	Task	Hint / what to test
☐	1	**Print "Hello, World!"**	Compile with gcc -Wall -Wextra -g. Read every warning.
☐	2	**Print your name, age, and city on separate lines**	Use \n and different format specifiers (%s, %d).
☐	3	**Read two integers; print sum, difference, product, quotient**	Check scanf's return value. Test division by 0. Try / vs % and int vs double.
☐	4	**Celsius to Fahrenheit**	F = C * 9/5 + 32. Why does 9/5 give 1 with ints?
☐	5	**Swap two variables (with and without a temp)**	Without temp: use +/- or XOR. Note the XOR trick fails if both are the same variable.
☐	6	**Print sizeof of char, int, long, float, double**	Print with %zu. Compare to limits.h (INT_MAX, etc.).
☐	7	**Area and circumference of a circle**	Use double and #define PI 3.14159265. Print with %.2f.
☐	8	**ASCII code of a char, and code back to char**	printf("%d %c", c, c). Print the whole table from 32 to 126.

Level 1: Conditions and Loops

Goal: control flow, logic, and number manipulation.

	#	Task	Hint / what to test
☐	9	**Even or odd**	n % 2. Test negatives (-3 % 2 is -1 in C).
☐	10	**Largest of three numbers**	Test equal values.
☐	11	**Leap year**	Divisible by 4, except by 100, unless by 400. Test 1900, 2000, 2024.
☐	12	**Calculator with switch (+ - * /)**	Handle divide by zero and an invalid operator in default.
☐	13	**Print 1 to N, then N down to 1**	Try for, while, and do-while versions.
☐	14	**Sum of digits**	Loop with n % 10 and n / 10. Test 0 and negatives.
☐	15	**Reverse a number (123 -> 321)**	Test 1200 (result 21). Watch for overflow.
☐	16	**Palindrome number**	Reuse your reverse function.
☐	17	**Multiplication table of N**	Format with %2d so columns align.
☐	18	**Factorial (unsigned long long)**	Find the largest N that does not overflow (20).
☐	19	**First N Fibonacci numbers**	Iterative first. Test N = 0, 1, 2.
☐	20	**Prime check, then all primes up to N**	Loop to sqrt(n). Then try the Sieve of Eratosthenes.
☐	21	**GCD and LCM**	Euclid: gcd(a,b) = gcd(b, a % b). lcm = a / gcd * b.
☐	22	**Count vowels, consonants, digits, spaces**	Read with getchar() until EOF (Ctrl+D).
☐	23	**Patterns: right triangle, pyramid, diamond, Pascal's triangle**	Think in rows: spaces then stars. Pascal: C(n,k) = C(n,k-1)*(n-k+1)/k.
☐	24	**Guess-the-number game**	srand(time(NULL)); rand() % 100 + 1. Give higher/lower hints and count attempts.

Level 2: Functions and Arrays

Goal: decompose code into functions, master indexing and 2D arrays.

	#	Task	Hint / what to test
☐	25	**Write max, abs_val, power**	Separate prototypes from definitions. Handle exp = 0 and negative exp.
☐	26	**Recursive factorial, Fibonacci, digit sum**	Always write the base case first. Note why recursive Fibonacci is slow.
☐	27	**Array: min, max, sum, average**	Average must be a double. Pass the size as a parameter.
☐	28	**Reverse an array in place**	Swap i and n-1-i up to the middle.
☐	29	**Linear search and binary search**	Binary search needs a sorted array. Test first, last, missing, and one element.
☐	30	**Bubble, selection, insertion sort**	Count comparisons and swaps in each. Test already sorted and reversed input.
☐	31	**Remove duplicates from a sorted array**	Two indices: read and write.
☐	32	**Rotate an array left by K**	Reversal trick: reverse(0,k-1), reverse(k,n-1), reverse(0,n-1). Use k % n.
☐	33	**Merge two sorted arrays**	Two pointers and a result array of size n+m.
☐	34	**Second largest element**	One pass, two variables. Handle duplicates and arrays with all-equal values.
☐	35	**Frequency of each element**	Counting array for small ranges. Then try for any range.
☐	36	**Matrix add, multiply, transpose**	Multiply needs a triple loop. Check dimensions match.
☐	37	**Is the matrix symmetric?**	Compare m[i][j] with m[j][i]. Must be square.
☐	38	**Spiral print of a matrix**	Track four boundaries: top, bottom, left, right.

Level 3: Strings (no string.h)

Goal: understand that a string is a char array ending in '\0'.

	#	Task	Hint / what to test
☐	39	**Write ft_strlen, ft_strcpy, ft_strcmp, ft_strcat**	Compare against the real ones with many inputs. strcmp returns the difference of the first differing bytes.
☐	40	**Reverse a string in place**	Do not try this on a string literal (it is read-only). Use char s[] = "abc";
☐	41	**String palindrome**	Then ignore case and non-letters.
☐	42	**Upper / lower case manually**	c - 'a' + 'A'. Then compare with toupper.
☐	43	**Count words in a sentence**	Handle multiple spaces, leading and trailing spaces.
☐	44	**Remove all spaces**	Read/write index technique, end with '\0'.
☐	45	**Anagram check**	Frequency table of 256 ints.
☐	46	**Your own atoi and itoa**	atoi: skip whitespace, handle sign, stop at first non-digit. Test INT_MIN.
☐	47	**First non-repeating character**	Two passes: count, then find.
☐	48	**Capitalize each word**	A letter after a space or at index 0 becomes uppercase, others lowercase.
☐	49	**Implement strstr**	Naive nested loop. Empty needle returns the haystack.

	#	Task	Hint / what to test
☐	50	**Split by delimiter**	Do this after Level 4. Returns char **, ends with NULL.

Level 4: Pointers and Dynamic Memory

Goal: pointers, malloc/calloc/realloc/free, no leaks, no crashes.

	#	Task	Hint / what to test
☐	51	**Swap two ints with pointers**	void swap(int *a, int *b). Draw the memory on paper.
☐	52	**Print an array with pointer arithmetic only**	*(p + i) and p++. Explain why arr[i] == *(arr + i).
☐	53	**Return multiple values via pointer params**	Example: min_max(arr, n, &min;, &max;).
☐	54	**malloc an array of size N from the user**	Check NULL, fill, print, free.
☐	55	**Same with calloc, then grow with realloc**	Use the temp-pointer pattern. Print to verify old data survived.
☐	56	**Write ft_strdup and ft_calloc**	strdup: len + 1. calloc: check count * size overflow, then memset.
☐	57	**Dynamic array (vector)**	struct { int *data; size_t len, cap; }. push, pop, get, free. Double capacity when full.
☐	58	**Allocate and free a 2D matrix (both ways)**	Rows of pointers vs a single block. Roll back on partial failure.
☐	59	**Read a line of unknown length**	Grow a buffer with realloc as you read with getchar/fgets.
☐	60	**ft_split returning char ** (plus free function)**	Count words, allocate, copy each. On failure free what you allocated so far.
☐	61	**Function pointers: add, sub, mul in one calculate()**	int calculate(int a, int b, int (*op)(int, int));
☐	62	**Sort with qsort and a custom comparator**	Sort ints ascending, descending, then strings. Never return a - b if it can overflow.
☐	63	**Create and catch a leak, use-after-free, double free**	Run with -fsanitize=address and valgrind. Read the reports carefully.

Level 5: Structs, Enums, Unions, Bits

Goal: model data, understand memory layout and bit manipulation.

	#	Task	Hint / what to test
☐	64	**struct student (name, age, grade); array of them**	Read, print, find best grade.
☐	65	**Sort students by grade, then name**	qsort with comparator on a struct. Tie-break with strcmp.
☐	66	**typedef, nested structs, by value vs by pointer**	Print sizeof of the struct. Compare copying cost and mutation behavior.
☐	67	**enum for days or state machine**	Use switch over the enum. Compile with -Wswitch to catch missing cases.
☐	68	**Union vs struct size**	Write a float, read the same memory as an int. Print sizeof of both.
☐	69	**Bit ops: set, clear, toggle, check a bit**	x |= 1u << n; x &= ~(1u << n); x ^= 1u << n; (x >> n) & 1.
☐	70	**Count set bits, power of two, XOR swap**	Kernighan: x &= x - 1 until 0. Power of two: x > 0 && (x & (x-1)) == 0.

	#	Task	Hint / what to test
☐	71	**Struct padding experiment**	Reorder fields and watch sizeof change. Try __attribute__((packed)).

Level 6: Data Structures

Goal: build classic structures from scratch, with correct memory handling.

	#	Task	Hint / what to test
☐	72	**Singly linked list: insert head/tail, delete, search, reverse**	Test empty list, one node, deleting head and tail. Write list_free.
☐	73	**Detect a cycle (Floyd)**	Slow and fast pointers.
☐	74	**Doubly linked list**	Update both prev and next on every insert/delete.
☐	75	**Stack (array, then linked list)**	push, pop, peek, is_empty. Handle underflow.
☐	76	**Queue and circular queue**	Use modulo for wrap-around. Distinguish full from empty.
☐	77	**Balanced parentheses with your stack**	Handle ( ) [ ] { } and wrong order like ([ )].
☐	78	**Binary search tree: insert, search, delete, traversals**	Delete has 3 cases: leaf, one child, two children (use in-order successor). Free recursively.
☐	79	**Hash table with chaining**	djb2 hash. Resize when load factor is high.
☐	80	**Min-heap and priority queue**	Sift up on insert, sift down on extract. Array representation: children at 2i+1 and 2i+2.
☐	81	**Graph (adjacency list): BFS and DFS**	BFS uses a queue, DFS uses recursion or a stack. Track visited.
☐	82	**Dijkstra's shortest path**	Use your heap. Test unreachable nodes.

Level 7: Files and I/O

Goal: read, write, and parse data robustly.

	#	Task	Hint / what to test
☐	83	**Write text to a file and read it back**	fopen, check NULL, fclose. Print strerror(errno) on failure.
☐	84	**Copy a file (simple cp)**	Read in chunks with fread / fwrite. Test with a large binary file.
☐	85	**Count lines, words, chars (wc)**	Compare your output with the real wc.
☐	86	**Last N lines of a file (tail)**	Simple: circular buffer of N lines. Advanced: fseek from the end.
☐	87	**Read a CSV into an array of structs**	Use fgets + strtok or your own split. Handle bad lines.
☐	88	**Binary file with fread / fwrite**	Save an array of structs and load it back. Note: padding and endianness.
☐	89	**Your own printf (%d %s %c %x)**	Use and write(1, ...). Handle NULL for %s and negative numbers.
☐	90	**get_next_line**	Static buffer between calls. Test with different BUFFER_SIZE (1, 42, 100000) and no trailing newline.

Level 8: Preprocessor, Build Tools, Multi-file

Goal: organize real projects and use professional tools.

	#	Task	Hint / what to test
☐	91	**Macros: MAX, SQUARE**	Show why SQUARE(1+2) breaks without parentheses, and why MAX(i++, j++) is dangerous.
☐	92	**Split a project into .h and .c files**	Include guards (#ifndef / #define / #endif). Declarations in .h, definitions in .c.
☐	93	**Write a Makefile (all, clean, fclean, re)**	Use variables, pattern rules (%.o: %.c), and header dependencies.
☐	94	**Build a static library**	ar rcs libmine.a *.o then gcc main.c -L. -lmine.
☐	95	**static and extern, scope and linkage**	Static global hides a symbol from other files. Static local keeps its value between calls.
☐	96	**Compile with -O2 and read assembly**	gcc -S -O0 vs -O2. Compare a simple loop.
☐	97	**Debug with gdb**	gdb ./a.out, break main, run, next, step, print var, bt, watch var.

Level 9: Systems Programming (Linux/Unix)

Goal: processes, signals, threads, sockets.

	#	Task	Hint / what to test
☐	98	**fork() and wait()**	Print PIDs in parent and child. Create several children.
☐	99	**exec to run ls**	execvp. If exec returns, it failed.
☐	100	**Two processes connected by pipe()**	Close unused ends. Parent writes, child reads.
☐	101	**Signals (SIGINT, SIGUSR1) with sigaction**	Only use async-signal-safe functions in handlers. Use volatile sig_atomic_t flags.
☐	102	**pthreads: create, join, fix a race with a mutex**	Two threads incrementing a shared counter 1,000,000 times. Show the wrong total, then fix it.
☐	103	**argc / argv and flag parsing**	Write a tiny tool with -v, -n 5, and a filename. Try getopt.
☐	104	**Environment variables**	getenv("PATH"), loop over environ.
☐	105	**TCP echo server and client**	socket, bind, listen, accept, recv, send. Test with nc localhost PORT.
☐	106	**A simple shell**	Prompt, parse, fork/exec, then add cd, exit, pipes |, redirections > < >>. Handle Ctrl+C and Ctrl+D.

Level 10: Hero Projects

Goal: bigger projects that combine everything. Pick at least three.

	#	Task	Hint / what to test
☐	107	**Your own libc subset**	strlen, memcpy, memmove, strchr, strtok, snprintf. Test against the real versions with random inputs.
☐	108	**Your own malloc and free**	Use sbrk or mmap. Keep a free list. Then add splitting, coalescing, and alignment.
☐	109	**A terminal game**	Tic-tac-toe with minimax, Snake with ncurses, or Minesweeper.
☐	110	**Tiny file-based database**	Insert, search, delete records, then add an index (hash or B-tree).

	#	Task	Hint / what to test
☐	111	**JSON or INI parser**	Tokenizer first, then recursive descent. Report error line numbers.
☐	112	**Simple HTTP web server**	Parse the request line, serve static files, return 404, then add threads.
☐	113	**Virtual machine or brainf*** interpreter**	Instruction array, program counter, stack. Precompute loop jump targets.
☐	114	**Mini compiler for arithmetic expressions**	Tokenizer, parser (precedence), AST, evaluator. Then generate bytecode.
☐	115	**Clone a Unix tool**	grep, ls, cat, diff, or tar. Compare behavior and output with the real one.

You made it

When every box is ticked, you will understand memory, pointers, data structures, files, processes, and tooling well enough to read and contribute to real C codebases. Keep going: study small open-source projects (for example, a Redis-like server, `cJSON`, or `sqlite` internals).